

EXP NO: 5

CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION WITH PCA

AIM

To implement K-Means clustering for grouping similar data points and Principal Component Analysis (PCA) for dimensionality reduction and visualization using a suitable dataset.

PROCEDURE

1. Import the required Python libraries.
2. Generate a suitable dataset using `make_blobs()`.
3. Apply the Elbow Method to determine the optimal number of clusters.
4. Apply K-Means clustering with the selected number of clusters.
5. Visualize the clusters and their centroids.
6. Calculate the Silhouette Score to evaluate the quality of clustering.
7. Generate a higher-dimensional dataset for PCA.
8. Standardize the dataset using `StandardScaler`.
9. Apply PCA to reduce the dimensionality of the dataset from 4 dimensions to 2 dimensions.
10. Calculate the explained variance ratio of the principal components.
11. Visualize the PCA-transformed data.
12. Apply K-Means clustering to the PCA-reduced data.
13. Visualize the clusters obtained from the PCA-reduced data.
14. Calculate the Silhouette Score for the PCA-based K-Means clustering.

CODE

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.datasets import make_blobs

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score
```

```
X, y = make_blobs(  
    n_samples=300,  
    centers=3,  
    cluster_std=0.60,  
    random_state=42  
)
```

```
df = pd.DataFrame(  
    X,  
    columns=["Feature_1", "Feature_2"]  
)
```

```
print(df.head())
```

```
wcss = []
```

```
for i in range(1, 11):  
    kmeans = KMeans(  
        n_clusters=i,  
        init="k-means++",  
        max_iter=300,  
        n_init=10,  
        random_state=42
```

```

    )
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.grid()
plt.show()

kmeans = KMeans(
    n_clusters=3,
    init="k-means++",
    max_iter=300,
    n_init=10,
    random_state=42
)

clusters = kmeans.fit_predict(X)

df["Cluster"] = clusters

```

```
print(df.head())
```

```
plt.figure(figsize=(8, 6))
```

```
sns.scatterplot(  
    x="Feature_1",  
    y="Feature_2",  
    hue="Cluster",  
    data=df,  
    palette="viridis",  
    s=100  
)
```

```
plt.scatter(  
    kmeans.cluster_centers_[:, 0],  
    kmeans.cluster_centers_[:, 1],  
    s=300,  
    marker="X",  
    label="Centroids"  
)
```

```
plt.title("K-Means Clustering")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.legend()
```

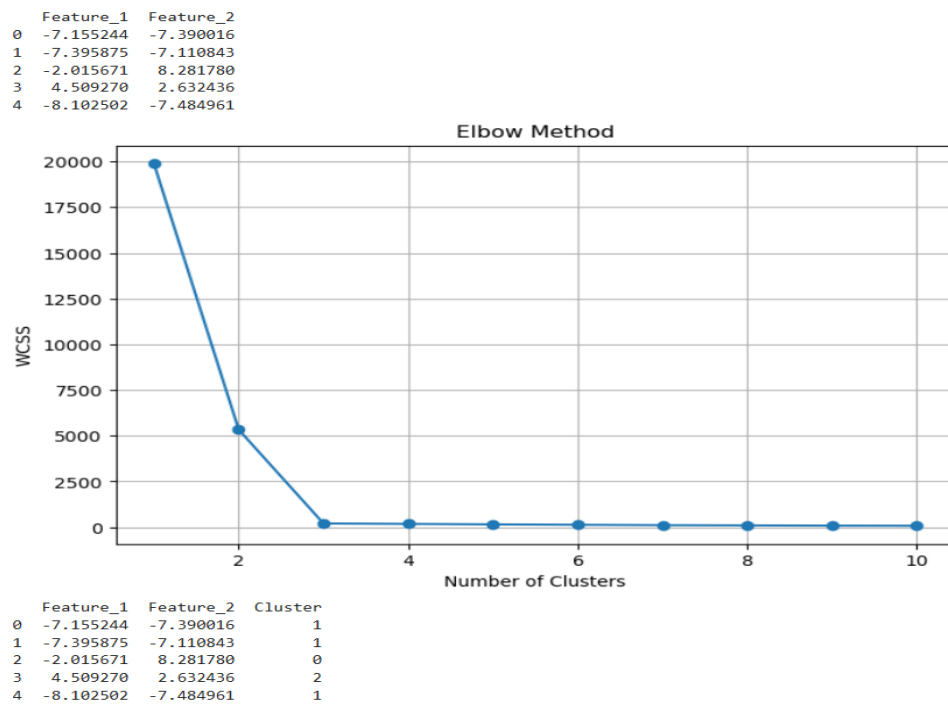
```
plt.grid()
```

```
plt.show()
```

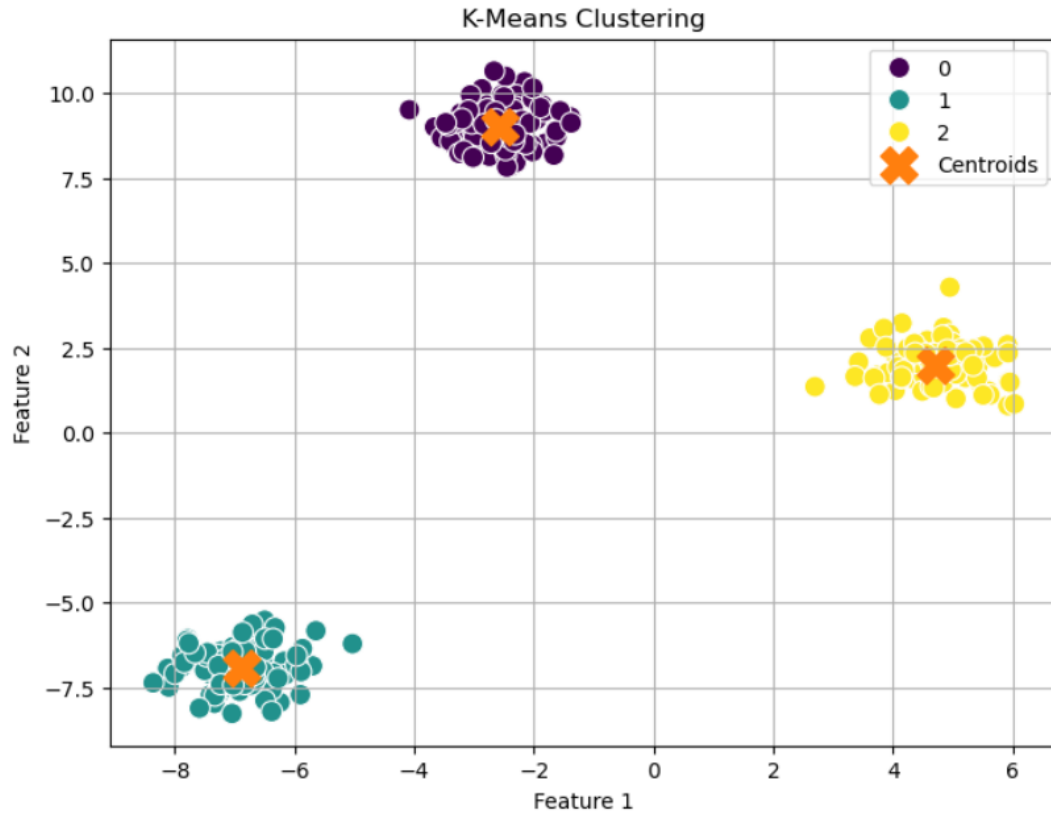
```
score = silhouette_score(X, clusters)
```

```
print("Silhouette Score:", round(score, 3))
```

OUTPUT



4 -0.102302 -7.404901 1



Silhouette Score: 0.908

RESULT

PCA was successfully applied to the four-dimensional dataset and reduced it to two principal components while retaining the major variance in the data. K-Means clustering was then performed on the PCA-reduced data, and the resulting clusters were visualized. The Silhouette Score was calculated to evaluate the clustering quality. Thus, PCA effectively simplified the high-dimensional data and K-Means successfully identified the underlying clusters.